`Remedition`

InvoicePortal - Remediation Documentation

1. Overview.

This document describes the vulnerabilities which present in the InvoicePortal lab and provides clear remediation steps to fix them.

2. Vulnerabilities Summary

#	Vulnerability	endpoint	
1	Weak JWT Signing Secret	/api/auth/login	High
2	Broken Object Level Authorization (BOLA)	/api/invoices, /api/invoices/{id}	Critical

3. Vulnerability 1: Weak JWT Signing Secret

Root causes

The application uses a weak, hardcoded JWT signing secret (`changeme2024`) that is:

- Present in a common wordlist
- Short and guessable
- Not rotated between environments

Because HS256 is symmetric, anyone who cracks the secret can forge valid tokens for any user.

Impact

- Full authentication bypass
- Ability to impersonate any user
- Complete compromise of the application's trust model

Remediation

- **Use strong, random secret**
- **Never ship default secrets**
- **Use asymmetric signing**
- **Add short token lifetimes and rotation**

4. Vulnerability 2: Broken Object Level Authorization (BOLA)

Root Causes

The `/api/invoices` and `/api/invoices/{id}` endpoints only verify that the JWT is valid, but do not check whether the authenticated user actually owns the requested invoice.

```
# Vulnerable code
rows = db.execute(
    "SELECT * FROM invoices WHERE owner_id = ?",
    (g.token_sub,)
).fetchall()
```

The application never compares `invoice.owner_id` against the authenticated user's ID before returning data.

Impact

- Any authenticated user can access any other user’s invoices
- Sensitive financial data exposed
- Complete tenant isolation failure

Remediation

1. Enforce ownership checks on every object-level query
    a. Modify the query to include the authenticated user's ID
2. Add authorization middleware
    a. Create a decorator that resolves the authentication user from the token and validated ownership
3. Use UUIDs plus ownership checks
    a. UUIDs are already used, which helps against enumeration. But UUIDs alone are not enough the ownership check is still required.

5. Verification and reset

After applying the fixes, verify with the tests:

login as attacker

```
curl -X POST http://localhost:5000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"attacker","password":"attacker_pass123"}'
```

Then attempt to access the victim's invoice with attacker's token

```
curl -X GET http://localhost:5000/api/invoices/<victim_invoic
e_id> \
  -H "Authorization: Bearer <attacker_token>"
```

Expected (Secure) : 403 Forbidden or 404 Not Found

Vulnerable Behavior: 200 with the victim's invoice data

verify that the JWT secret is not crackable

Run hashcat against a fresh token with the same wordlist. The secret should not appear.

confirm that the forged tokens are rejected

Attempt to forge token with the guessed secret the server should reject it with 401 unauthorized

6. Summary

Vulnerability	Fix
Weak JWT Secret	Use strong random secret, switch to RS256, enforce rotation
BOLA	Add ownership checks on all object-level queries